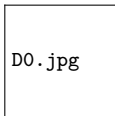


Lecture 13: Tuple Relational Calculus (TRC)

Declarative Querying and Logical Foundations

Z2004: Database Management Systems



IIT Madras Zanzibar

Friday, 27 March 2026

1. The Shift: Procedural vs. Declarative

Relational Algebra (Procedural)

Relational Algebra tells the system exactly **HOW** to get the data. It is a step-by-step recipe.

The Taxi Analogy:

"Drive 200 meters forward, take a left at the light, merge onto the highway, take exit 4..."

Relational Calculus (Declarative)

Relational Calculus tells the system exactly **WHAT** data you want, and lets the DBMS figure out how to get it.

The Taxi Analogy:

"Take me to the Airport."

Two Types of Calculus

- ▶ **Tuple Relational Calculus (TRC):**
Variables represent entire rows (tuples).
This is what SQL is primarily based on!
- ▶ **Domain Relational Calculus (DRC):**
Variables represent individual columns (domains). *(Next Lecture!)*

The Goal

Today, we learn how to write mathematical requests that SQL engines use as their foundational logic.

2. A Quick Primer on the Math Symbols

The Core Symbols

TRC uses symbols from Set Theory and Logic.

- ▶ **{ }** (**Curly Braces**): Defines a mathematical **Set**. TRC queries always return sets of rows.
- ▶ **|** (**Vertical Bar**): Read as "**Such That**". Separates the output (left) from the rules (right).
- ▶ **∈** (**Element Of**): Read as "**Belongs To**".
Example: $t \in User$ means the row t is inside the *User* table.

Logic & Quantifiers

- ▶ $\wedge$ (AND) $\vee$ (OR) $\neg$ (NOT)
- ▶ $\exists$ (There Exists) $\forall$ (For All)

Putting It Together

Let's translate a formula:

$$\{ t \mid t \in User \wedge t.Verified = 'True' \}$$

 "Return the **set** of all rows t , **such that** t belongs to the *User* table, **AND** t 's *Verified* status is 'True'."

2. TRC Syntax: The General Form

The Core Formula

$$\{t \mid P(t)\}$$

 Read as: "The set of all tuples t such that condition P is true for t ."

The Variables

- ▶ t (**Tuple Variable**): It ranges over all tuples of some relation. Think of t as a cursor scanning through rows in a table.
- ▶ $\mid$ (**Such That**): The divider between the output (left) and the condition (right).

The Predicate & Result

- ▶ $P(t)$ (**Predicate**): A formula that must evaluate to *true* for a tuple to appear in the result.
- ▶ **Result**: The final set of all tuples t that successfully satisfy $P(t)$.

2. TRC Syntax: Atoms & Connectives

Atoms in $P(t)$

An **atom** is the smallest mathematical building block of a formula. It must evaluate to either True or False.

- ▶ $t \in User$
Membership: Tuple t is a row in the User table.
- ▶ $t.UID \theta s.UID$
Attribute Comparison: Comparing columns between two tuples (e.g., $t.UID = s.UID$).
- ▶ $t.Views \theta c$
Constant Comparison: Comparing a column to a fixed value (e.g., $t.Views > 1000$).

Comparison Operators (θ):

$\theta \in \{=, \neq, <, \leq, >, \geq\}$

Logical Connectives

Connectives are used to combine multiple atoms into a complex condition.

- ▶ $P_1 \wedge P_2$ (Logical AND)
- ▶ $P_1 \vee P_2$ (Logical OR)
- ▶ $\neg P$ (Logical NOT)

Quantifiers

- ▶ $\exists s \in R (P(s))$
There exists at least one tuple s in relation R such that $P(s)$ is true.
- ▶ $\forall s \in R (P(s))$
For ALL tuples s in relation R , the condition $P(s)$ is true.

The Running Schema: Social Platform

User

 UID — Handle — Verified

Like

 UID —  PID — Date

Post

 PID — Topic — Views

User(UID, Handle, Verified)

Like(UID, PID, Date)

Post(PID, Topic, Views)

Sample Data Used in Examples

User has rows with UID U1–U5; Handles like @coder, @gamer; Verified 'True'/'False'.

Post P100 = “Database Memes”, P101 = “Java Tips”.

3. Basic TRC Queries (Select & Project)

Example 1: Filtering Rows

Goal: Get all data for **Verified** users on the platform.

$$\{t \mid t \in User \wedge t.Verified = 'True'\}$$

SQL Translation:

```
SELECT * FROM User t
WHERE t.Verified = 'True';
```

Notice how the TRC formula maps almost perfectly to SQL!

Example 2: Filtering Columns

Goal: Get **only the Handles** (usernames) of Verified users.

$$\{t.Handle \mid t \in User \wedge t.Verified = 'True'\}$$

💡 How Projection Works

In TRC, you perform a Projection by specifying exactly which attribute you want **on the left side** of the vertical bar ($\mid$).

If you want the whole row, you just write t . If you want just the Handle, you write $t.Handle$.

4. The Logical Quantifiers ($\exists$ and $\forall$)

The Existential Quantifier ($\exists$)

Meaning: "There exists at least one..."

We use $\exists$ when we need to link to another table (like a Join) to prove an interaction happened.

Goal: Handles of users who *Liked* post 'P100'.

$\{u.Handle \mid u \in User \wedge$
 $\exists L \in Like (L.UID = u.UID \wedge L.PID = 'P100')\}$

 "Give me the user's handle IF there exists a Like record from this user for post P100."

The Universal Quantifier ($\forall$)

Meaning: "For ALL..."

We use $\forall$ when dealing with absolute requirements (the calculus equivalent of the Algebra Division $\div$ operator).

Goal: Superfans (Users who liked **every single post** on the platform).

$\{u.Handle \mid u \in User \wedge$
 $\forall p \in Post (\exists L \in Like (L.UID = u.UID \wedge L.PID = p.PID))\}$

 "For ALL posts, there exists a Like linking this user to that post."

4b. Intermediate TRC: Set Difference (NOT)

The "NOT" Operator ($\neg$)

To perform a **Set Difference** (finding records that do *not* have a match), we combine the logical NOT ($\neg$) with the existential quantifier ($\exists$).

Goal: Find the Handles of users who have **NEVER** liked a post.

$$\{u.Handle \mid u \in User \wedge \neg(\exists L \in Like (L.UID = u.UID))\}$$

 "Return the user's handle IF there does NOT exist any Like record matching their UID."

SQL Equivalent (NOT EXISTS)

In SQL, this mathematical concept maps perfectly to the NOT EXISTS subquery clause.

```
SELECT u.Handle
FROM User u
WHERE NOT EXISTS (
  SELECT 1
  FROM Like L
  WHERE L.UID = u.UID
);
```

Common Mistake:

Do not write $L.UID \neq u.UID$. You must assert that an exact match *does not exist anywhere* in the table!

4c. Intermediate TRC: Multi-Condition Joins

Intersection (Logical AND)

What if a user must meet **two separate conditions** from the same related table? You must instantiate the tuple variable *twice*!

Goal: Find users who liked **BOTH** post 'P100' and 'P101'.

$$\{u.Handle \mid u \in User \wedge (\exists L1 \in Like (L1.UID = u.UID \wedge L1.PID = 'P100')) \wedge (\exists L2 \in Like (L2.UID = u.UID \wedge L2.PID = 'P101'))\}$$

Why Two Variables? ($L1$ & $L2$)

A single tuple (row) in the Like table can only hold **one** PID at a time. It cannot be 'P100' and 'P101' simultaneously!

Therefore, we must prove the existence of **two distinct rows** ($L1$ and $L2$) for the same user.

SQL Equivalent:

```
SELECT u.Handle FROM User u
JOIN Like L1 ON u.UID = L1.UID
JOIN Like L2 ON u.UID = L2.UID
WHERE L1.PID = 'P100'
      AND L2.PID = 'P101';
```

5. Safety & Codd's Theorem

The Danger of Unsafe Queries

Because TRC is pure math, you can write a formula that produces an **infinite** output!

$$\{t \mid \neg(t \in User)\}$$

"Give me every entity in the universe that is NOT a registered User." This would return infinite, meaningless garbage data!

The Safety Rule:

A TRC query is **Safe** if it restricts its output to only values that physically exist within the database tables mentioned in the formula.

Codd's Theorem (1972)

Dr. Edgar F. Codd proved that:

Relational Algebra $\equiv$ Safe TRC $\equiv$ Safe DRC

Why this changed the world:

- ▶ They are all **equally expressive**.
- ▶ This proved that you can build a user-friendly declarative language (SQL) and a computer can flawlessly translate it into procedural math (Algebra) to execute it.
- ▶ This is the foundation of the modern Database Optimizer.

Class Activity: TRC Translation

A New Database Schema

Consider a new Livestreaming database to test your knowledge:

Streamer Table

SID	Username	Game
 S1	Ninja	Fortnite
 S2	TenZ	Valorant

Subscriber Table

SubID	Username	SID
 U1	GamerKid	 S1
 U2	ProSniper	 S2

Instructions:

Take 5 minutes to solve the 2 tasks on the right.

The Tasks

Task 1: Read the Calculus

- Translate the following TRC expression into plain English. What is it asking for?

$\{s.Username \mid s \in \text{Streamer} \wedge s.Game = 'Valorant'\}$

Task 2: Write the Calculus

- Write the TRC expression to find the **Username**s of all **Subscribers** who are subbed to **Ninja**.
- *Hint: You will need to use the Existential Quantifier ($\exists$) to link the Subscriber to the Streamer!*

Class Activity: Solutions

Task 1 Solution

The Query:

$\{s.Username \mid s \in \text{Streamer} \wedge s.Game = 'Valorant'\}$

Plain English Translation:

"Retrieve the **Username**s of all streamers whose game is **Valorant**."

SQL Equivalent:

```
SELECT Username
FROM Streamer
WHERE Game = 'Valorant';
```

Task 2 Solution

The Prompt:

Find the **Username**s of all **Subscribers** who are subbed to **Ninja**.

The TRC Expression:

$\{sub.Username \mid sub \in \text{Subscriber} \wedge$
 $\exists st \in \text{Streamer} (st.SID =$
 $sub.SID \wedge st.Username = 'Ninja')\}$

Breakdown:

- ▶ $sub.Username$: We only want the subscriber's name.
- ▶ $\exists st \in \text{Streamer}$: We must look inside the Streamer table to find a match.

Key Takeaways

- ✓ **Declarative Logic:** TRC specifies *what* data is required, not the steps to get it.
- ✓ **TRC Syntax:** $\{t \mid P(t)\}$. The variable t represents an entire row.
- ✓ **Quantifiers:** $\exists$ links tables (like joins). $\forall$ handles absolute inclusion (like division).
- ✓ **Safety:** Queries must only generate data that exists in referenced tables to avoid infinite outputs.
- ✓ **Codd's Theorem:** Relational Algebra and Relational Calculus have equal mathematical power.

Next Lecture (L14)

Domain Relational Calculus (DRC)

- ➔ Shifting variables from full rows to individual columns (domains).
- ➔ DRC Syntax and examples.
- ➔ Converting TRC to DRC.
- ➔ Intro to QBE (Query By Example).

Reading Assignment

- 📖 Read **Silberschatz, Ch. 6** (Domain Relational Calculus) to prepare.